Recursion-2

Agenda:

1. Fibonacci number,
2. Fast power function
3. Count of element k in the Array
4. Indices of element k in Array

1. Nth fibonacci

→ fibonacci seq. starts with 0 and 1, and every subsequent number is the sum of the two preceding ones.

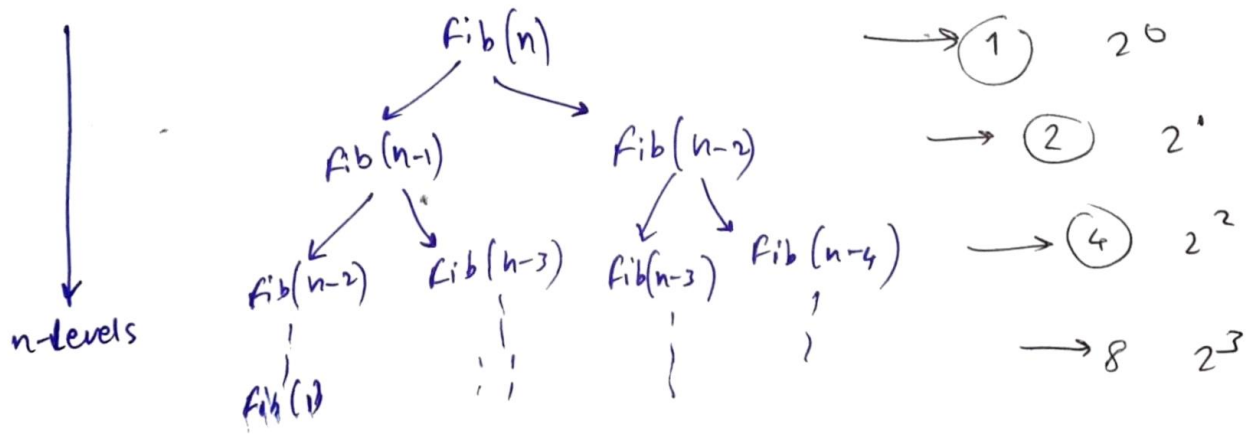
$$f[n] = f[n-1] + f[n-2]$$

code →

```
public int fib(int n) {
    // Base Case: f(0) = 0 , f(1) = 1
    if (n <= 1) { return n; }

    // Recursive Call
    return fib(n-1) + fib(n-2);
}
```

- **Complexity** → **T.C**: $O(2^n)$ because no. of calls doubles at almost every level of recursion tree



$$2^0 + 2^1 + 2^2 \text{ ---- } + 2^{n-1} = 2^n - 1$$

$$\boxed{T.C \leq O(2^n)}$$

S.C: $O(N)$ due to depth of the recursion stack

② Fast Power Function

↳ goal is to compute A^B efficiently. Instead of multiplying A by itself B times (which takes $O(B)$ time), we use the property of exponents.

ex: $3^4 = 81$, $2^5 = 32$

Code →

```
public long Fast Power(int A, int B){
    if (B==0) return 1;
    long half power = fast Power (A, B/2);
    if (B % 2 ==0){
        return half Power * halfPower; }
    else {
        return A * halfPower * halfpower;
    }
}
```

logic →

$A^B \rightarrow A^{B/2} * A^{B/2}$, if B is even

$A^B \rightarrow A * A^{(B-1)/2} * A^{(B-1)/2}$, if B is odd.

$2^{30} \rightarrow 2^{15} \times 2^{15}$

$2^{15} \rightarrow 2 \times 2^7 \times 2^7$

$2^7 \rightarrow 2 \times 2^3 \times 2^3$

$2^3 \rightarrow 2 \times 2^1 \times 2^1$

$2^1 \rightarrow 2 \times 2^0 \times 2^0$

$2^0 \rightarrow 1$

↳ 6 steps

Dry Run:

```
fast (3,20) {
    a2 = fast (3, 10){          // 59049
        a2 = fast(3,5) {        // 243
            a2 = fast (3,2) {   // 9
                a2 = fast (3,1) {   // 3
                a2 = fast (3,0){    // 1
                    return 1
                }
                return 3*1*1 = 3
            }
            return 3*3 = 9
        }
        return 3*9*9 = 243
    }
    return 243 * 243 = 59049
}
return 59049 * 59049
}
```

- T.C ⇒ $O(\log_2 B)$
- S.C ⇒ $O(N)$

$B \rightarrow \frac{B}{2} \rightarrow \frac{B}{2^2} \cdots \frac{B}{2^k} \rightarrow 2^k = B \rightarrow k = \log_2 B$

(3) # Count of Element K in the Array

Q: Task counts how many times a target integer K appears in an array without using loop.

Code →

```
public int countK (int[] A, int i, int K){
    if (i == A.length){
        return 0;
    }
    if (A[i] == K){
        return 1 + countK (A, i+1, K); }
    else {
        return countK (A, i+1, K);
    }
}
```

count (A, 0, K)
↓
count (A, 1, K)
⋮
count (A, A.length, K) N

* Function calls = N+1

T.C = O(N) S.C = O(N)

(4) # Indices of element K an Array

Q Given an array of integer A[] with N elements and a target integer B, the task is to find all the indices at which B occurs in the array.

$A = [\overset{0}{4}, \overset{1}{5}, \overset{2}{3}, \overset{3}{1}, \overset{4}{5}, \overset{5}{4}, \overset{6}{5}]$ → B = 5 → Ans. [1, 4, 6]

Code →

```
public int[] solve (int[] A, int i, int B, int count){
    if (i == A.length){
        return new int [count];
        // create arr of size 'count' at the end
    if (A[i] == B){
        int[] result = solve (A, i+1, B, count+1);
        result [count] = i;   // store current index
        return result;
    }
    else {
        return solve (A, i+1, B, count);
    }
}
```